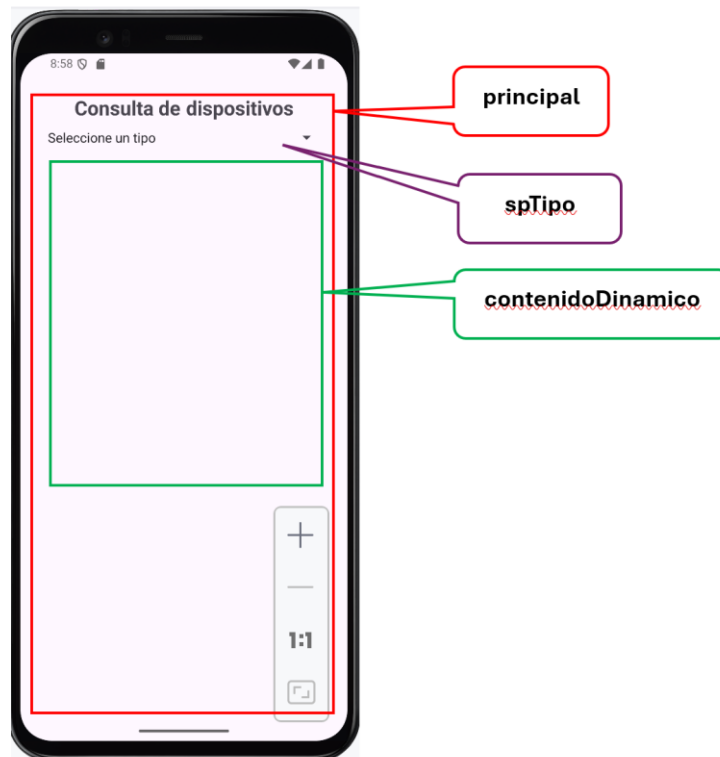


Unidad 6 Clase 3: Creación de elementos y control de eventos desde el Controller

En esta clase, vamos a desarrollar una app para consultar información de distintos dispositivos inteligentes, que puede ser: móviles, tablets o smartwatches.

Al principio se mostrará en la actividad (pantalla/vista) la lista de los tipos de dispositivos disponibles. Esta actividad tendrá inicialmente:

- Un `LinearLayout` para distribuir los views verticalmente con id **principal**.
Dentro de este `viewgroup` se encuentra:
 - un `textView` con el título,
 - un `spinner` con id **spTipo**
 - un `LinearLayout` con id **contenidoDinamico**



Se le provee la actividad creada en el archivo xml, las clases del modelo y el controller con cierto código ya agregado.

Usted debe realizar las siguientes actividades en el controller MainActivity.java:

1. Crear las variables de instancia para el spinner, y el linearLayout

Método onCreate

2. En el método **onCreate** recuperar los ids del spinner y del linearLayout y asignarlos a las variables antes declaradas.
3. Agregar un listener para el spinner **spTipo** para que cuando se seleccione un tipo de dispositivo se invoque al método **agregarSegundoSpinner(position)** que muestra los diferentes modelos de ese tipo. El parámetro position indica el índice de elemento seleccionado (empieza siempre en 0). Validar que position no sea 0 porque el primer elemento no es válido (solo dice Seleccione).

NOTA:

Considerar que no siempre necesita le índice, en otras ocasiones podría recuperar todo el objeto seleccionado con

MiObjeto selectedItem = (MiObjeto) spTipo.getSelectedItem();

```
spTipo.setOnItemClickListener(new
AdapterView.OnItemClickListener() {
    // establece que se realizará cuando se capture el evento de
    elemento seleccionado en el spinner
    @Override
    public void onItemClick(AdapterView<?> parent, View view,
int position, long id) {
        if (position != 0) {

            agregarSegundoSpinner(position); // se cargará el
segundo spinner con sus datos y se agregara al linearlayout
contenidoDinamico
        }
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) {
        // Do nothing
    }
});
```

Método agregarSegundoSpinner

Este método recibe el índice del tipo seleccionado en el spinner **spTipo**.

4. Remover todos los views del linearLayout **contenidoDinamico**
5. Crear un nuevo spinner **newSpinner**.
6. Crear un ArrayAdapter para este spinner. Considerar que en este adapter se podrá cargar cualquier lista de objetos que sean **DispositivoInteligente**.

Para crear un ArrayAdapter se usa la sintaxis:

```
ArrayAdapter<MiObjeto> adapter = new ArrayAdapter<>(this,  
android.R.layout.simple_list_item_1, listaMiObjeto);
```

7. Agregar el adapter al spinner **newSpinner**.
8. Agregar el **newSpinner** al linearLayout referenciado por la variable **contenido**.
9. Agregar un listener para el spinner **newSpinner** para que cuando se seleccione un modelo de dispositivo del **newSpinner** se invoque al método **llenarDatos(int indiceSelpRol, int indiceSelnewSpinner)** que recibe el índice del seleccionado en el primer spinner **spTipo** y el índice del elemento seleccionado en el **newSpinner**.

Método llenarDatos

10. Remover todos los views del linearLayout **contenidoDinamico** excepto el primero porque el primero es el **newSpinner**
11. Crear dos **TextViews** para mostrar la información de los dispositivos.
12. Crear un **imageView** para mostrar las imágenes de los dispositivos.
13. Configurar las propiedades del **ImageView**.

```
imageView.setLayoutParams(new LinearLayout.LayoutParams(  
    LinearLayout.LayoutParams.WRAP_CONTENT,  
    LinearLayout.LayoutParams.WRAP_CONTENT  
));  
imageView.setScaleType(ImageView.ScaleType.CENTER_CROP);
```

14. Crear un **Button** y setear el texto Salir. No necesita ajustar el texto en Strings previamente, porque está creando el botón en el controller directamente.
15. Configurar las propiedades del **Button para agregar diseño**.

```
// Configurar las propiedades del botón
LinearLayout.LayoutParams layoutParams = new
LinearLayout.LayoutParams(
    LinearLayout.LayoutParams.WRAP_CONTENT, // Ancho del botón
    se ajusta al contenido
    LinearLayout.LayoutParams.WRAP_CONTENT // Alto del botón
    se ajusta al contenido
);
layoutParams.gravity = Gravity.CENTER; // Centrar el botón
horizontalmente

buttonSalir.setLayoutParams(layoutParams);
```

16. Para agregar diseño al botón que se creó desde el controller, hay que:

- a. Crear un archivo XML de diseño, con el nombre que desee en este caso **rounded_button**, ubicado en la carpeta drawable:

```
// res/drawable/rounded_button.xml
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item>
        <shape android:shape="rectangle">
            <solid android:color="#800080" /> <!-- Color morado -->
            <corners android:radius="16dp" /> <!-- Bordes redondeados -->
        </shape>
    </item>
</selector>
```

- b. Aplicar el diseño al botón desde el controlador

```
buttonSalir.setBackground(ContextCompat.getDrawable(this,
R.drawable.rounded_button));
```

17. Para establecer el color del texto del botón:

```
buttonSalir.setTextColor(Color.WHITE);
```

18. Ajustar el padding de los elementos:

textView1.setPadding(16,36,0,16);

19. Agregar textos a los **TextViews** dependiendo del tipo de dispositivo (Móvil, Tablet, Smartwatch). Usar un switch para esto.

Si es un Movil, se mostrarán: Nombre y Sistema Operativo.

Si es una Tablet, se mostrarán: Nombre y Tamaño.

Si es un Smartwatch, se mostrarán: Nombre y si tiene GPS

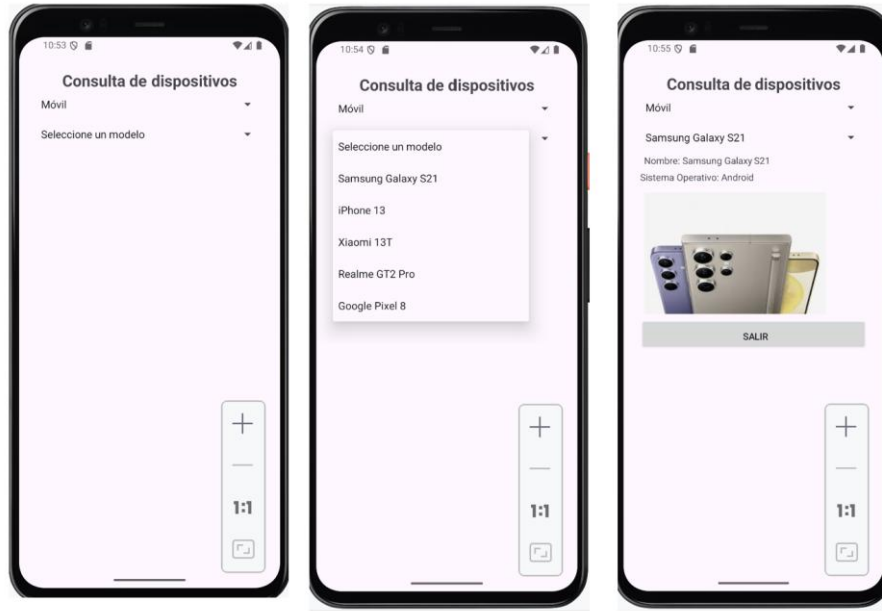
20. Setear la imagen que se mostrará en el **imageView** dependiendo del tipo de dispositivo (Móvil, Tablet, Smartwatch). Las imágenes ya deben estar en la carpeta Drawable. Se invocan solo con el nombre del archivo, sin la extensión.

```
imageView.setImageResource(R.drawable.tab1);
```

21. Agregar los **TextViews**, el **imageView** y el **Button** al **linearLayout** **contenidoDinamico** representado por la variable **contenido**.

22. Agregar un listener al botón para que cuando se dé clic en él se finalice la aplicación.

```
buttonSalir.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        finish(); ////programar lo que se espera que suceda cuando se  
        da clic en el botón  
    }  
});
```



Agregar una imagen desde la galería del dispositivo

Para esto necesitamos agregar en el `LinearLayout` con id principal:

Un `Button` con id `btnCargarImagen`.

Un `ImageView` con id `imagenSeleccionada`.

Un `Button` con id `Un Button con id btnGuardar`.

Un `Button` con id `Un Button con id btnMostrarListado`.

1. Crear las variables de instancia:

```
private Button btnCargarImagen;
private ImageView imagenSeleccionada;
private static final int REQUEST_IMAGE_PICK = 100;
private ActivityResultLauncher<Intent> imagePickerLauncher;
```

2. Inicializarla en el método `onCreate`

```
imagePickerLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK && result.getData()
            != null) {
            Uri imageUri = result.getData().getData();
            imagenSeleccionada.setImageURI(imageUri);
        }
    })
```

```
    }  
});
```

3. Agregar un listener al botón para que cuando se dé clic en él se acceda a la galería para elegir la imagen a cargar, y la imagen se muestre en el ImageView.

```
btnCargarImagen.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        Intent intent = new Intent(Intent.ACTION_PICK);  
        intent.setType("image/*");  
        imagePickerLauncher.launch(intent);  
    }  
});
```